

You are here



Showcase



Volume Number: 13 (1997)

Issue Number: 5

Column Tag: Programmer's Challenge

Programmer's Challenge

By Bob Boonstra, Westford, MA

Equation Evaluator

Those of you with PowerPCs have probably experimented with the Graphing Calculator application installed by default into the Apple Menu Items folder. As one of the first native applications for the PowerPC, the ability of the Graphing Calculator to display, and even animate 2-dimensional and 3-dimensional equations demonstrating the computing power of the PowerPC chip using native code. Underlying the display capability is code to parse an equation and rapidly compute the equation value for a range of input values. In this month's Challenge, you will produce code to perform these parsing and computation functions.

The prototype for the code you should write is:

```
typedef unsigned long ulong;

typedef struct Values {
    float first;    /* first equation input value */
    float delta;    /* first+delta is second input value */
    ulong howMany; /* number of input values */
} Values;

typedef struct IntValues {
    long first; /* first equation input value */
    long delta; /* first+delta is second input value */
    ulong howMany; /* number of input values */
} IntValues;

typedef struct Results {
    float equationResult; /* result of equation given x,y,n */
    float x; /* input value of x producing equationResult */
    float y; /* input value of x producing equationResult */
    long n; /* input value of x producing equationResult */
} Results;

void Evaluate(
    char *equation, /* null-terminated equation to evaluate */
    const Values *xP, /* equation input values for x */
    const Values *yP, /* equation input values for y */
    const IntValues *nP, /* equation input values for n */
    Results w[] /* preallocated storage for equation values */
);
```

The input equation you are to evaluate will be provided as a null-terminated string in the 2-dimensional curve form ($y=x+2x^2$) or the 3-dimensional surface form ($z=r \cos(t) r \sin(t)$). You are to evaluate the equation for each of the values of x and n (in the case of a 2-dimensional equation) or x , y , and n (in the 3-dimensional case) provided and return the results in the preallocated array w . Each input is described as an initial value, a delta between each value and the next value, and the number $howMany$ of values this input parameter is to assume. For example, if x is to take on the range of values $[1.0, 1.1, 1.2, 2.0]$, then the x input could be described as:

THIS MONTH'S ISSUE

TODAY'S DEAL

```
xP->first = 1.0; xP->delta = .1; xP->howMany = 11
```

In the event that the equation does not contain one of the input parameters, that parameter should be ignored. There is no guarantee, for example, that nP->howMany will be zero when the input equation is not a function of n. Similarly, for a 2-dimensional equation, yP should be ignored.

The input equation might be written as a function of r and q, which should be calculated from x and y just as the Graphing Calculator does. Because the Graphing Calculator displays equations in ways that cannot be directly represented in a character string, the following symbols will be used in the equation to represent the operator or value indicated:

- \ square root
- / division
- ^ exponentiation
- p pi (p)
- t theta (q)
- . multiplication (also represented by juxtaposition)

Standard algebraic operator precedence should be used: exponentiation and square roots, then multiplication and division, then addition and subtraction, with left-to-right evaluation order for operators of equal precedence, and with parentheses used to override normal precedence. Arguments to trigonometric functions will always be surrounded by parentheses. Where the Graphing Calculator would use superscripts to indicate an extended exponent, parentheses will be used to make the meaning clear (e.g., e^(x+y)).

Store the equation result for the i-th combination of input values in w[i]->equationResult, and store the corresponding input values in w[i]->x, w[i]->y, and w[i]->n. The results may be stored in any order, but each input combination should appear exactly once. Results should be calculated with a minimum relative accuracy of .00001.

Even though the Graphing Calculator handles inequalities, multiple equations, differentiation, simplification, and expansion, your code does not need to deal with these cases. With these exceptions, your code should handle any equation that the Graphing Calculator can handle.

You may allocate any amount of dynamic storage up to 20MB, provided you deallocate the storage before returning. This will be a native PowerPC Challenge, using the CodeWarrior environment. Solutions may be coded in C, C++, or Pascal. Limited use of assembly language is also permitted, for anyone who might need it to access any dynamically generated code as part of their solution. The solution computing the correct results in the least amount of time will be the winner.

Three Months Ago Winner

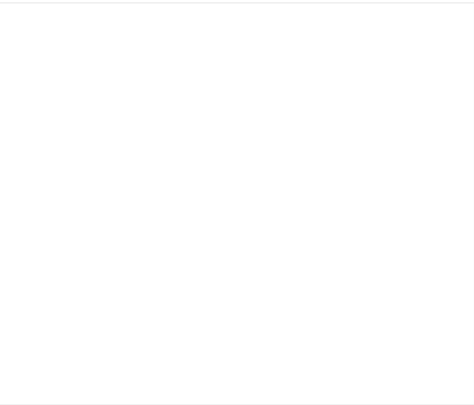
Congratulations to **Jeff Mallett** (Boulder Creek, CA) for producing the winning entry among ten submitted for the Othello Challenge. The objective was to win a round-robin Othello tournament, generalized to allow a board size between 8 and 64 (even numbers only), by as large a margin as possible, using as little computation time as possible. Tournament points were based on the margin of victory (the number of the winner's pieces showing at the end of the game minus the number of the opponent's pieces showing), and on the amount of computation time used, as follows:

[margin of victory - seconds of execution time / 30] / number of squares

The test cases included board sizes of 8x8 and 18x18, with each player competing against each other player twice, once playing first, with the black pieces, and once playing second, with the white pieces. I had planned to run some larger cases, but the first two still had not completed after running all night, so I had to stop at 18x18.

The solutions submitted varied considerably in complexity. The simplest (and fastest) solutions assigned values to each board position and made the move which maximized total value. Some solutions took that approach one step further and evaluated an opponent's potential responses and used a min/max approach to select the best move. Other solutions took credit for the number of pieces flipped on a move, with possible consideration for vulnerability to reversal on the next move. The winning solution used the most complicated approach, with lines of play being examined by a progressively deepening search.

The table below describes how each player fared against each other player. Each row shows the number of victories that player had against the player represented by the corresponding column. The final column shows the total number of victories won out of the 36 games played by each entry. As you can see, the second place entry by **Randy Boring** won nearly as many games as Jeff's winning entry, and actually beat the winning entry in one of the four games they played.



Search

?

Community Search:

GO

MacTech Search:

GO

Software Updates via MacUpdate

Latest Forum Discussions

SEE ALL

iPHONE APP SHOWCASE

Price Scanner via MacPrices.net

Jobs Board

Player Name	1	2	3	4	5	6	7	8	9	10	Wins
1 David Whitney	-	1	4	2	4	2	2	0	0	4	19
2 Dan Farmer	3	-	4	2	4	4	4	1	0	4	26
3 David McGavran	0	0	-	2	4	1	1	0	1	1	10
4 Eric Hangstefer	2	2	2	-	4	0	1	0	0	3	14
5 Ken Slezak	0	0	0	0	-	0	0	0	0	0	0
6 Gregory Cooper	2	0	3	4	4	-	2	0	1	4	20
7 Mason Thomas	2	0	3	3	4	2	-	0	0	3	17
8 Randy Boring	4	3	4	4	4	4	4	-	1	4	32
9 Jeff Mallett	4	4	3	4	4	3	4	3	-	4	33
10 Ludovic Nicolle	0	0	3	1	4	0	1	0	0	-	9

The top two entries used significantly more computation time than the others. Jeff's winning entry used an average of more than one second per move (as you can see by examining the parameter settings in his code), but the larger margin of victory more than offset the execution time penalty.

The table below provides the code and data sizes for each of the solutions submitted, along with the total number of victories won in all of the test cases, and the cumulative score earned in those victories. Numbers in parenthesis after a person's name indicate that person's cumulative point total for all previous Challenges, not including this one.

Player	Code	Data	Wins	Score
Jeff Mallett (44)	6988	277	33	25.49
Randy Boring (27)	6908	64	32	20.37
Gregory Cooper (27)	4764	284	20	15.00
Dan Farmer	9240	96	26	14.27
David Whitney	7216	864	19	9.79
Eric Hangstefer	4124	80	14	9.72
Mason Thomas (4)	6976	57	17	9.01
David McGavran	3272	48	10	8.09
Ludovic Nicolle (21)	6436	120	9	6.33
Ken Slezak (20)	9256	77	0	0.00

Sample Game

Here is one of the games played by the top two entries. Randy Boring's entry played Black, and Jeff Mallett's played White. The moves are given as the row and column selected by the programs, interspersed with an occasional view of the resulting board position.

Move Black (Boring) White (Mallett)

	row	col	row	col
1	2	3	4	2
2	5	2	2	4
3	2	5	3	5
4	4	5	3	6
5	2	2	3	2
6	2	6	5	3

- - - - -
- - - - -
- - B B B B B -
- - W W W B W -
- - W W B B - -
- - B W - - - -
- - - - -
- - - - -

731

...missing an opportunity to place a piece on the edge at (2,7), inviting (1,7) as a response by white, in hope of moving to (0,7)

54
84115
937

- - - - -
- - - - - W - -
- - B B B W B -
- B B B B B B B
- B B B B W - -

```

- - B W W - - -
- - - - - - -
- - - - - - -

```

Black occupies the first edge square.

```

      27
101713
111246
120355
136202

```

```

- - W B - - - -
- - B W - W - B
- - B B W W B B
- B B W B B W B
- B B W B W W -
- - B B W W - -
- - B - - - - -
- - - - - - -

```

Black's next move, to (0,1), prevents White from obtaining a foothold on one of the edges.

```

140121
153020
161050
175163
180440
196072
201416

```

```

- B B B B - - -
B - B B B B W B
B W B B B W W B
B B B W B B W B
B B W W B W W -
B B W W W W - -
B - W W - - - -
- - W - - - - -

```

While white had other choices, none of them were very good, and this move to (1,6) gives Black a corner on the next move.

```

210761
2270

```

Black takes a second corner ...

```

- B B B B - - B
B - B B B B B B
B W B B B B W B
B W B W B B W B
B W W B B W W -
B W B W W W - -
B B W W - - - -
B - W - - - - -

```

```

      71
237374
246564
257566
2677

```

and a third

```

- B B B B - - B
B - B B B B B B
B W B B B B W B
B B B W B B W B
B W B B B B W -
B B W B B B - -
B W B W W W B -
B B B B B B - B

```

```

      56

```

277667
2811

- B B B B - - B
B B B B B B B
B B B B B B W B
B B B W B B W B
B W B B B B W -
B B W W B W W -
B W B W W W W W
B B B B B B B B

Black could have done better with (5,7) at this point, instead of giving the final corner to White.

00
294757
30- - 05
3106

W W W W W B B
B W B B W B B B
B B W W B W W B
B B W W B B B B
B W B B B W B B
B B W W B W W W
B W B W W B W W
B B B B B B B B

Black wins by a score of 37 to 27, a relatively close game compared to many in the tournament.

TOP 20 CONTESTANTS

Here are the point totals for the Top Contestants in the Programmer's Challenge. The numbers below include points awarded over the 24 most recent contests, including points earned by this month's entrants.

Rank	Name	Points	Rank	Name	Points
1.	Munter, Ernst	184	11.	Cutts, Kevin	21
2.	Gregg, Xan	114	12.	Nicolle, Ludovic	21
3.	Larsson, Gustav	47	13.	Picao, Miguel Cruz	21
4.	Lengyel, Eric	40	14.	Brown, Jorg	20
5.	Boring, Randy	37	15.	Gundrum, Eric	20
6.	Cooper, Greg	34	16.	Higgins, Charles	20
7.	Lewis, Peter	32	17.	Kasparian, Raffi	20
8.	Mallett, Jeff	27	18.	Slezak, Ken	20
9.	Antoniewicz, Andy	24	19.	Studer, Thomas	20
10.	Beith, Gary	24	20.	Karsh, Bill	19

There are three ways to earn points: (1) scoring in the top 5 of any Challenge, (2) being the first person to find a bug in a published winning solution or, (3) being the first person to suggest a Challenge that I use. The points you can win are:

1st place	20 points	5th place	2 points
2nd place	10 points	finding bug	2 points
3rd place	7 points	suggesting Challenge	2 points
4th place	4 points		

Here is Jeff's winning solution:

Jello.c An nxn Othello program in C

*Copyright 1997 Jeff Mallett

```
/*
 * Uses alpha-beta search with iterative deepening, transposition tables,
 * extension for solving, futility cut-off, a simplistic selectivity, etc.
 */
```

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MY_ASSERT(b)

Engine Parameters

    // Extensions/Pruning
#define kFullDepthPlies 2 // # of full-depth plies before selectivity
#define kSolveThreshold 4 // Plies extension for solving whole board
#define kMinimumSafeDisks3 // Minimum disks to have and still be safe
#define kFutilityScore kCorner
    // Score above beta to trigger futility cut-off
#define kWipeOutExtension 2
    // Extend a ply if opponent doesn't have more disks than this

    // Average move time in 60ths of a second
#define kOpeningMoveTime 50 // In the opening
#define kMoveTime 150 // Normally
#define kSolveMoveTime 800 // When trying to solve

    // Scoring Parameters
#define kFinalDisk50 // per disk on board at the end
#define kTooFewDisks-50 // per disk under threshold

Constants/Macros
#define kInfinity1100000000L
#define kBestPossible100000000L

// Directions (if se increases both x and y):
// 7 6 5 4 3 2 1 0
// NE SW NW SE N S W E (opposites are adjacent)
enum { DIR_E = 0, DIR_W, DIR_S, DIR_N,
        DIR_SE, DIR_NW, DIR_SW, DIR_NE };
enum { E = 0x0001,
        W = 0x0002,
        S = 0x0004,
        N = 0x0008,
        SE = 0x0010,
        NW = 0x0020,
        SW = 0x0040,
        NE = 0x0080,

        E_BORDER = 0x0100,
        W_BORDER = 0x0200,
        S_BORDER = 0x0400,
        N_BORDER = 0x0800,
        SE_BORDER = 0x1000,
        NW_BORDER = 0x2000,
        SW_BORDER = 0x4000,
        NE_BORDER = 0x8000
};

// Array of squares on the board (up to 66 x 66 squares)
// Each square has:
// 1st 8 bits - In this direction there's a BLACK disk that can be flipped by WHITE
#define ADJACENCY0x000000FF // Adjacent to disk in 8 directions
#define BORDER_ADJACENCY 0x0000FF00 // Adjacent to border in 8 directions
#define WHITE 0x00010000 // Is White disk here?
#define BLACK 0x00020000 // Is Black disk here?
#define COLOR_BITS 0x00030000 // Mask: Is disk here?
#define BORDER_BIT 0x00040000 // Is this border square?
#define COLOR_BORDER_BITS 0x00070000 // Mask: Is border or disk here?
#define BAD_BIT 0x00080000 // X or C square
#define EMPTYADJ_BITS0xFFFF0000 /* Top 12 bits (0-4095) */

#define BORDER_ADJACENCY_SHIFT8
#define COLOR_SHIFT16
#define EMPTYADJ_SHIFT 20

```

```

#define IS_NOT_EMPTY(z)    ((z) & COLOR_BORDER_BITS)
#define IS_EMPTY(z)!IS_NOT_EMPTY(z)
#define HAS_DISK(z)((z) & COLOR_BITS)
#define HAS_NO_DISK(z)    !HAS_DISK(z)
#define IS_BORDER(z) ((z) & BORDER_BIT)
#define IS_ON_BOARD(z)    !IS_BORDER(z)
#define IS_BAD(z)((z) & BAD_BIT)
#define IS_EDGE(z) ((z) & BORDER_ADJACENCY)
#define IS_CORNER(z) (gCountZeros[((z) & BORDER_ADJACENCY) \
    >> BORDER_ADJACENCY_SHIFT] == 3)

#define OPPONENT(side)    ((side) ^ COLOR_BITS)

#define XY2INDEX(x, y)    ((y) * gRealBoardSize + (x))

#define COUNT_EMPTYIES(z) \
    gCountZeros[ ((z) | ((z) >> \
    BORDER_ADJACENCY_SHIFT)) & ADJACENCY ]

#define DIR_BIT(dir) (1L << (dir))
#define OPP_DIR_BIT(dir)  gOppDirBit[dir]

#define EMPTYADJ_BIT(index) \
    ((index) << EMPTYADJ_SHIFT)
#define GET_EMPTYYADJ_INDEX(pSq) \
    (*(pSq) >> EMPTYADJ_SHIFT)
#define SET_EMPTYYADJ_INDEX(pSq, index) \
    *(pSq) = (*(pSq) & ~EMPTYADJ_BITS) | EMPTYADJ_BIT(index)

// Add to end of list
#define ADD_TO_EMPTYYADJ(pSq) \
    SET_EMPTYYADJ_INDEX(pSq, gSizeEmptyAdj); \
    gEmptyAdj[gSizeEmptyAdj++] = pSq

// Swap entry in list with end entry and shrink list
#define REMOVE_FROM_EMPTYYADJ(pSq) { \
    long i = GET_EMPTYYADJ_INDEX(pSq); \
    unsigned long *p = gEmptyAdj[-gSizeEmptyAdj]; \
    gEmptyAdj[i] = p; \
    SET_EMPTYYADJ_INDEX(p, i); \
}

#define PUSH(x) *(gChangesEnd++) = (x)
#define START_SAVE PUSH(0L)
#define PUSH_SQ(pSq) \
    { PUSH(*(pSq)); PUSH((unsigned long)(pSq)); }
#define POP *(-gChangesEnd)
#define TOP *gChangesEnd

typedef unsigned long * PSQUARE;
typedef long SCORE;

#define USE_HASH
#ifdef USE_HASH
#define kHashTableMask    0x7FFF // 32K entry table
#define kHashTableSize    (kHashTableMask + 1)
#define kSwitchSideHash    0x87654321
enum { INVALID = 0, VALID = 1,
    LOWER_BOUND = 2, UPPER_BOUND = 3 };

typedef struct SHash {
    unsigned long HashValue;
    SCORE Score;
    PSQUARE BestMove;
    short Depth;
    short Type;
} SHash;

static SHash *gHashTable;
static long gWhiteHashOffset, gBlackHashOffset,
    gFlipHashOffset;
static unsigned long gHashValue;
#endif

```

Prototypes

```
Boolean /*legalMove*/ Othello (
    long boardSize, /* number of rows/columns in the game board */
    long oppRow, /* row where opponent last moved, 0 .. boardSize-1 */
    long oppCol, /* column where opponent last moved, 0 .. boardSize-1 */
    long *moveRow, /* return your move - row, 0 .. boardSize-1 */
    long *moveCol, /* return your move - column, 0 .. boardSize-1 */
    void *privStorage, /* preallocated storage for your use */
    long storageSize, /* size of privStorage in bytes */
    Boolean newGame, /* TRUE if this is your first move in a new game */
    Boolean playBlack /* TRUE if you play first (black pieces) */
);
```

```
static SCORE Search(SCORE alpha, SCORE beta, unsigned long side, long
depth, Boolean passEndsGame);
static long Generate(unsigned long side);
static SCORE MakeMove(PSQUARE to, unsigned long side);
static void UnmakeMove();
static void Initialize(long boardSize, void *privStorage);
static SCORE Evaluate(unsigned long side);
static long SortValue(PSQUARE p, unsigned long side);
static void BubbleSort(long n, unsigned long side);
```

Static Variables

```
// Direction indices
```

```
static unsigned long gOppDirBit[8] =
    { W, E, N, S, NW, SE, NE, SW };
```

```
// Offsets on board plus zero sentinel
```

```
// Fill in gOffsets[2..7] when we know board size
```

```
static long gOffsets[9] = {1L,-1L,99L,99L,99L,99L,99L,99L,0L};
```

```
// gSquares - Array of squares: Stores board
```

```
static unsigned long *gSquares;
```

```
static unsigned long *gOnBoardStart; // Pointer into gSquares
```

```
static unsigned long *gOnBoardEnd; // Pointer into gSquares
```

```
static long gNumOnSquares; // # of squares, not including borders
```

```
static long gRealBoardSize; // Length of a side (includes borders)
```

```
// gEmptyAdj - Array of pointers to squares:
```

```
// Stores all empty squares adjacent to disk(s)
```

```
static PSQUARE *gEmptyAdj;
```

```
static long gSizeEmptyAdj;
```

```
// gChanges - Array of unsigned longs containing data to undo moves
```

```
// list of:
```

```
// <pointer to square> <old square value>
```

```
// terminated by a 0L
```

```
// The first position will be the drop square and the others will be flips
```

```
static unsigned long *gChanges;
```

```
static unsigned long *gChangesEnd; // Pointer into gChanges
```

```
// gCountZeros - Precalculated 256-element constant array
```

```
// returns count of zero bits in the byte
```

```
static unsigned long *gCountZeros;
```

```
// gTree - Array of pointers to squares: Holds search tree
```

```
static PSQUARE *gTree;
```

```
static PSQUARE *gTreeEnd; // Pointer into gTree
```

```
// gMobility - Array of counts of moves available at each ply
```

```
static long *gMobility;
```

```
// Pointers to various special squares
```

```
static PSQUARE gNWCorner, gNWx, gNWC1, gNWC2;
```

```
static PSQUARE gNECorner, gNEX, gNEC1, gNEC2;
```

```
static PSQUARE gSWCorner, gSWx, gSWC1, gSWC2;
```

```
static PSQUARE gSECorner, gSEX, gSEC1, gSEC2;
```

```
// The gCounts array holds current counts of disks
```

```
// Access is by gCounts[side >> COLOR_SHIFT]
```

```
// gCounts[WHITE_INDEX] white's disks
```

```
// gCounts[BLACK_INDEX] black's disks
```

```

#define WHITE_INDEX(WHITE >> COLOR_SHIFT)
#define BLACK_INDEX(BLACK >> COLOR_SHIFT)
static unsigned long gCounts[3];

static SCORE gIncScore;    // Score relative to side to move
static SCORE gEndgameDiskBonus;    // Bonus per disk in endgame
static SCORE kX, kC, kEdge, kCorner; // Penalties/Bonuses

static long gAbortSearchTime; // Time at which the search will be aborted
static Boolean gAborted;    // Has the search been aborted?

static long gStartDepth;    // Search was started at this depth
static long gPly;           // Number of moves deep

Othello
Boolean /*legalMove*/ Othello (
    long boardSize,    /* number of rows/columns in the game board */
    long oppRow,        /* row where opponent last moved, 0 .. boardSize-1 */
    long oppCol,        /* column where opponent last moved, 0 .. boardSize-1 */
    long *moveRow,      /* return your move - row, 0 .. boardSize-1 */
    long *moveCol,      /* return your move - column, 0 .. boardSize-1 */
    void *privStorage, /* preallocated storage for your use */
    long storageSize,  /* size of privStorage in bytes */
    Boolean newGame,    /* TRUE if this is your first move in a new game */
    Boolean playBlack   /* TRUE if you play first (black pieces) */
)
{
    PSQUARE *to, p;
    unsigned long side = playBlack ? BLACK : WHITE;
    unsigned long nextSide;
    SCORE t, bestScore, saveIncScore;
    long j, generated, index, x, y;
    long stillOpen, bestFoundAt, *pOffsets;
    long startTime, targetTime, targetDuration;
    Boolean nearEdge;
#ifdef USE_HASH
    unsigned long saveHashValue;
#endif

    if (newGame) {
        Initialize(boardSize, privStorage);
    }
    gChangesEnd = gChanges;

#ifdef USE_HASH
    // Fix up gHashTable
    {
        SHash *pHashTable = gHashTable;
        int i;

        i = kHashTableSize - 1;
        do {
            (pHashTable++)->Depth -= 2;
        } while (i--);
    }
#endif

    if (oppRow != -1) {
        index = XY2INDEX(oppCol+1, oppRow+1);
        gIncScore -=
            MakeMove(&gSquares[index], playBlack ? WHITE : BLACK);
        gChangesEnd = gChanges;
    }

    gTreeEnd = gTree;
    generated = Generate(side);

    if (!generated) { // No moves
        *moveRow = *moveCol = -1;
        return TRUE;
    }
    if (generated > 1 && !(newGame && oppRow == -1)) {
        BubbleSort(generated, side);
    }

```

```

gMobility[0] = gMobility[1] = generated;
gPly = 1;

nextSide = OPPONENT(side);
stillOpen = gNumOnSquares - gCounts[WHITE_INDEX] -
gCounts[BLACK_INDEX];

    // Calculate gEndgameDiskBonus
gEndgameDiskBonus = 0;
x = gNumOnSquares / 3;
j = x - stillOpen;
if (j > 0) {
gEndgameDiskBonus = (j * kFinalDisk) / (x * 5);
if (!gEndgameDiskBonus)
gEndgameDiskBonus = 1;
}

    // Do we have any pieces near an edge?
nearEdge = false;
for (p = gOnBoardStart;
!nearEdge && p <= gOnBoardEnd; ++p) {
if (HAS_DISK(*p)) {
if (IS_EDGE(*p)) {
nearEdge = true;
} else {
pOffsets = gOffsets;
do {
if (IS_EDGE(*(p + *pOffsets)))
nearEdge = true;
} while (!nearEdge && *(++pOffsets));
}
}
}

    // Set times
if (!nearEdge)
targetDuration = kOpeningMoveTime;
else if (stillOpen > 20)
targetDuration = kMoveTime;
else // try to solve!
targetDuration = kSolveMoveTime;
startTime = LMGetTicks();
targetTime = startTime + targetDuration;
gAbortSearchTime = startTime + targetDuration * 6;
gAborted = false;

saveIncScore = gIncScore;
#ifdef USE_HASH
saveHashValue = gHashValue;
#endif

for (gStartDepth=1; gStartDepth < stillOpen && !gAborted;
++gStartDepth) {
to = gTree;
bestScore = -kInfinity;
for (j=0; j<generated; ++j) {
gIncScore = - (gIncScore + MakeMove(*to, side));
++gPly;
t = -Search(-kInfinity, kInfinity, nextSide,
gStartDepth - 1, false);
--gPly;
UnmakeMove();
gIncScore = saveIncScore;
#ifdef USE_HASH
gHashValue = saveHashValue;
#endif
if (gAborted)
break;

if (t > bestScore) {
PSQUARE bestMove, *p;
// Move *to to front of the tree

```

```

bestMove = *to;
MY_ASSERT(bestMove >= gOnBoardStart && bestMove <=
gOnBoardEnd);

for (p = to; p != gTree; -p)
*p = *(p-1);
*gTree = bestMove;

bestScore = t;
bestFoundAt = gStartDepth;
}
if (LMGetTicks() >= targetTime) {
if (bestScore > -kInfinity && gStartDepth +
kSolveThreshold + 1 != stillOpen)
break; // time to stop
if (LMGetTicks() - startTime
>= 3 * targetDuration)
break; // time to stop
}
++to;
}

if (gStartDepth >= 4 && LMGetTicks() - startTime > 1 +
targetDuration/2)
break; // probably not enough time to finish another iteration
if (gStartDepth - bestFoundAt == 3
&& IS_CORNER(*gTree[0]))
break; // easy corner move
}
}

gIncScore += MakeMove(*gTree, side);
index = (long)(*gTree - gSquares);
y = index / gRealBoardSize;
x = index - y * gRealBoardSize;
*moveCol = x - 1;
*moveRow = y - 1;

return TRUE;
}

```

SortValue

// Returns value that orders squares for root search

long SortValue(PSQUARE p, unsigned long side)

```

{
    long stillOpen, value;
    PSQUARE q;
    unsigned long occupant, enemy;
    long *pOffsets;

```

```

    if (IS_EDGE(*p)) {
        if (IS_CORNER(*p))
            return 200; // Corner
        if (IS_BAD(*p))
            return -100; // C
        return 100; // Edge
    }

```

```

    if (IS_BAD(*p))
        return -200; // X

```

```

    // Check p's adjacency bits
    stillOpen = gNumOnSquares - gCounts[WHITE_INDEX] -
gCounts[BLACK_INDEX];
    enemy = OPPONENT(side);
    value = 0;
    pOffsets = gOffsets;
    do {
        q = p + *pOffsets;
        occupant = *q & COLOR_BITS;
        if (stillOpen > 10) {
            if (occupant == side)
                ++value; // good to take away empty-adjacent
            else if (occupant == enemy)

```

```

    -value; // bad to flip
  } else if (occupant == OPPONENT(side))
    ++value; // endgame: good to flip
  } while (*(++pOffsets));

  return value;
}

```

BubbleSort

// Sorts generated root moves in decreasing value order

// Hey, bubble sort is really okay in this case

void BubbleSort(long n, unsigned long side)

```

{
    long i, j, swaps;
    PSQUARE temp;

    for (j=n-2; j>=0; -j) {
        swaps = 0;
        i = 0;
        do {
            if (SortValue(gTree[i+1], side)
                > SortValue(gTree[i], side)) {
                ++swaps; // Swap i and i+1
                temp = gTree[i];
                gTree[i] = gTree[i+1];
                gTree[i+1] = temp;
            }
        } while (i++ != j);
        if (!swaps)
            break; // Already sorted: Finish early
    }
}

```

Evaluate

// Evaluate position and return score relative to side

// side is also the side to move

SCORE Evaluate(unsigned long side)

```

{
    SCORE score = 0;

    // Maximize disks, but only in endgame
    if (gEndgameDiskBonus) {
        score = (gCounts[WHITE_INDEX] - gCounts[BLACK_INDEX]) * gEndgameDiskBonus;
        if (side == BLACK)
            score = -score;
    }

    // NW Corner Area
    if (HAS_DISK(*gNWCorner)) {
        score += (*gNWCorner & side) ? kCorner : -kCorner;
    } else if (*gNWCorner & ADJACENCY) {
        if (HAS_DISK(*gNWX)) {
            score += (*gNWX & side) ? kX : -kX;
        }
        if (HAS_DISK(*gNWC1)) {
            score += (*gNWC1 & side) ? kC : -kC;
        }
        if (HAS_DISK(*gNWC2)) {
            score += (*gNWC2 & side) ? kC : -kC;
        }
    }

    // NE Corner Area
    if (HAS_DISK(*gNECorner)) {
        score += (*gNECorner & side) ? kCorner : -kCorner;
    } else if (*gNECorner & ADJACENCY) {
        if (HAS_DISK(*gNEX)) {
            score += (*gNEX & side) ? kX : -kX;
        }
        if (HAS_DISK(*gNEC1)) {
            score += (*gNEC1 & side) ? kC : -kC;
        }
    }
}

```

```

    if (HAS_DISK(*gNEC2)) {
        score += (*gNEC2 & side) ? kC : -kC;
    }
}

// SW Corner Area
if (HAS_DISK(*gSWCorner)) {
    score += (*gSWCorner & side) ? kCorner : -kCorner;
} else if (*gSWCorner & ADJACENCY) {
    if (HAS_DISK(*gSWX)) {
        score += (*gSWX & side) ? kX : -kX;
    }
    if (HAS_DISK(*gSWC1)) {
        score += (*gSWC1 & side) ? kC : -kC;
    }
    if (HAS_DISK(*gSWC2)) {
        score += (*gSWC2 & side) ? kC : -kC;
    }
}

// SE Corner Area
if (HAS_DISK(*gSECorner)) {
    score += (*gSECorner & side) ? kCorner : -kCorner;
} else if (*gSECorner & ADJACENCY) {
    if (HAS_DISK(*gSEX)) {
        score += (*gSEX & side) ? kX : -kX;
    }
    if (HAS_DISK(*gSEC1)) {
        score += (*gSEC1 & side) ? kC : -kC;
    }
    if (HAS_DISK(*gSEC2)) {
        score += (*gSEC2 & side) ? kC : -kC;
    }
}

// Too few disks?
if (gCounts[WHITE_INDEX] < kMinimumSafeDisks) {
    SCORE x = (kMinimumSafeDisks - gCounts[WHITE_INDEX]) *
    kTooFewDisks;
    score += (side == WHITE) ? x : -x * 2;
}
if (gCounts[BLACK_INDEX] < kMinimumSafeDisks) {
    SCORE x = (kMinimumSafeDisks - gCounts[BLACK_INDEX]) *
    kTooFewDisks;
    score += (side == BLACK) ? x : -x * 2;
}

// Mobility
score += gMobility[gPly-2] - gMobility[gPly-1];

// Could also have a value for the right to move

return gIncScore + score;
}

Search
SCORE Search(SCORE alpha, SCORE beta, unsigned long side, long depth,
Boolean passEndsGame)
{
    register PSQUARE *to;
    unsigned long nextSide;
    long generated;
    SCORE t, bestScore, saveIncScore;
    PSQUARE *firstMove;
    long stillOpen;
    SCORE oldAlpha = alpha;
#ifdef USE_HASH
    unsigned long saveHashValue;
    PSQUARE bestMove, tableReply;
    SHash *pHashTable;
#endif

    stillOpen = gNumOnSquares - gCounts[WHITE_INDEX] -
    gCounts[BLACK_INDEX];

```

```

    if (!stillOpen) {
        // Board full
        bestScore = (gCounts[WHITE_INDEX] -
            gCounts[BLACK_INDEX]) * kFinalDisk;
        if (side == BLACK)
            bestScore = -bestScore;
        return bestScore;
    }

    if (!gCounts[WHITE_INDEX]) {
        // White is wiped out!
        bestScore = kBestPossible;
        if (side == WHITE)
            bestScore = -kBestPossible;
        return bestScore;
    }

    if (!gCounts[BLACK_INDEX]) {
        // Black is wiped out!
        bestScore = kBestPossible;
        if (side == BLACK)
            bestScore = -kBestPossible;
        return bestScore;
    }

    if (depth <= 0) {
        if (stillOpen > kSolveThreshold &&
            gCounts[OPPONENT(side) >> COLOR_SHIFT]
            > kWipeOutExtension) {
            bestScore = Evaluate(side);
#ifdef USE_HASH
            bestMove = NULL;
#endif
            goto HashSave; // Terminal node
        }
        else if (depth == 1 && stillOpen > kSolveThreshold) {
            t = Evaluate(side);
            if (t > beta + kFutilityScore)
                return t; // Futility cut-off
        }

#ifdef USE_HASH
        tableReply = NULL;
        pHashTable = &gHashTable[gHashValue & kHashTableMask];
        if (pHashTable->HashValue == gHashValue) {
            tableReply = pHashTable->BestMove;
            if (pHashTable->Depth >= depth) {
                if (pHashTable->Type == VALID) {
                    if (pHashTable->Score > beta)
                        alpha = beta;
                    else if (pHashTable->Score > alpha)
                        alpha = pHashTable->Score;
                }
                else if (pHashTable->Type == LOWER_BOUND) {
                    if (pHashTable->Score > beta)
                        return beta + 1;
                }
                else if (pHashTable->Type == UPPER_BOUND) {
                    if (pHashTable->Score < alpha)
                        return alpha - 1;
                }
            }
            if (alpha > beta)
                return beta;
        }
        else if (pHashTable->Type == LOWER_BOUND) {
            if (pHashTable->Score > beta)
                return beta + 1;
        }
        else if (pHashTable->Type == UPPER_BOUND) {
            if (pHashTable->Score < alpha)
                return alpha - 1;
        }
    }
    if (alpha > beta)
        return beta;
}

// Abort?
if (LMGetTicks() >= gAbortSearchTime) {
    gAborted = true;
    return 0;
}

#ifdef USE_HASH
    bestMove = NULL;

```

```

#endif
nextSide = OPPONENT(side);

firstMove = gTreeEnd;
generated = Generate(side);
gMobility[gPly] = generated;

if (!generated) { // no moves available
if (passEndsGame) {
bestScore = (gCounts[WHITE_INDEX] -
gCounts[BLACK_INDEX]) * kFinalDisk;
if (side == BLACK)
bestScore = -bestScore;
return bestScore;
}
#ifdef USE_HASH
gHashValue ^= kSwitchSideHash;
#endif
gIncScore = -gIncScore;
++gPly;
bestScore = -Search(-beta, -alpha,
nextSide, depth, true);
-gPly;
gIncScore = -gIncScore;
++depth;
goto Searched;
}

to = firstMove;
bestScore = -kInfinity;
#ifdef USE_HASH
if (tableReply && *to != tableReply) {
// Find tableReply in move list and move to front
PSQUARE *p;
for (p = to + 1; *p; ++p)
if (*p == tableReply) {
// Swap *p and *to
*p = *to;
*to = tableReply;
break;
}
}
#endif

saveIncScore = gIncScore;
#ifdef USE_HASH
gHashValue ^= kSwitchSideHash;
saveHashValue = gHashValue;
#endif

do {
if (!IS_BAD(**to) || // selectivity
#ifdef USE_HASH
!bestMove ||
#endif
gStartDepth - depth <= kFullDepthPlies ||
stillOpen <= kSolveThreshold) {
gIncScore = - (gIncScore + MakeMove(*to, side));
++gPly;
t = -Search(-beta, -alpha, nextSide, depth-1, false);
-gPly;
UnmakeMove();
gIncScore = saveIncScore;
#ifdef USE_HASH
gHashValue = saveHashValue;
#endif
if (gAborted) {
#ifdef USE_HASH
bestMove = NULL;
#endif
break;
}
}

```

```

    if (t > bestScore) {
#ifdef USE_HASH
        bestMove = *to;
#endif
        if (t > alpha) {
            if (t >= beta) {
                bestScore = t;
                break;
            }
            alpha = t;
        }
        bestScore = t;
    }
    ++to;
} while (*to);

Searched: ;
#ifdef USE_HASH
    gHashValue ^= kSwitchSideHash;
#endif
gTreeEnd = firstMove;

#ifdef USE_HASH
    if (bestMove) {
#endif
HashSave: ;
#ifdef USE_HASH
    pHashTable = &gHashTable[gHashValue & kHashTableMask];
    if (pHashTable->Depth <= depth) {
        pHashTable->HashValue = gHashValue;
        pHashTable->Depth = depth;
        pHashTable->Score = bestScore;
        pHashTable->BestMove = bestMove;
        MY_ASSERT(!bestMove || IS_EMPTY(*bestMove));

        pHashTable->Type = VALID;
        if (bestScore <= oldAlpha) {
            pHashTable->Type = UPPER_BOUND;
        } else if (bestScore >= beta) {
            pHashTable->Type = LOWER_BOUND;
        }
    }
}
#endif

    return bestScore;
}

Generate
#define ADD_MOVE(pSq)*(gTreeEnd++) = pSq
long Generate(unsigned long side)
{
    PSQUARE *e, p, q, *afterCorners, *movesStart, lastBad;
    unsigned long enemy;
    long i, *pOffsets;

    enemy = OPPONENT(side);
    afterCorners = movesStart = gTreeEnd;
    lastBad = NULL;

    e = gEmptyAdj;
    i = gSizeEmptyAdj;
    while (i-) {
        p = *e;
        MY_ASSERT(IS_EMPTY(*p) && (*p & ADJACENCY));
        pOffsets = gOffsets;
        do {
            q = p + *pOffsets;
            if (*q & enemy) {
                do { // Skip through line of enemy disks
                    q += *pOffsets;
                } while (*q & enemy);
            }
        } while (q & enemy);
    }
}

```

```

    if (*q & side) { // Add square p to move list!
        // Bad? Try to put it on the end
        if (IS_BAD(*p)) {
            if (!lastBad) {
                lastBad = p;
                break;
            }
            ADD_MOVE(p);
            break;
        }
        if (!IS_CORNER(*p)) {
            // Add to end after corners
            ADD_MOVE(p);
            break;
        }
        // Corner: keep all corners on front
        if (afterCorners == gTreeEnd) { // All are corners!
            ADD_MOVE(p);
        } else {
            unsigned long *temp = *afterCorners;
            *afterCorners = p;
            ADD_MOVE(temp);
        }
        ++afterCorners;
        break;
    }
    } while (*(++pOffsets));
    ++e;
}
if (lastBad) {
    ADD_MOVE(lastBad);
}
ADD_MOVE(NULL); // sentinel
return (long)(gTreeEnd - movesStart) - 1;
}

MakeMove
// Makes the move for "side" on the "to" square
// Saves the move to gChanges for later undo'ing
// Returns the change in score relative to the moving side
SCORE MakeMove(PSQUARE to, unsigned long side)
{
    unsigned long z;
    PSQUARE q, p;
    long dir, offset;
    long flips = 0;
    SCORE score = 0;
    unsigned long enemy = OPPONENT(side);

    MY_ASSERT(IS_EMPTY(*to) && (*to & ADJACENCY));

#ifdef USE_HASH
    // Update hash for new disk
    if (side == BLACK)
        gHashValue ^= *(to + gBlackHashOffset);
    else
        gHashValue ^= *(to + gWhiteHashOffset);
#endif

    START_SAVE;

    // Do flipping, Update adjacency
    dir = 7;
    do {
        offset = gOffsets[dir];
        q = to + offset;
        if (IS_ON_BOARD(*q)) {
            if (IS_EMPTY(*q)) {
                if ( !(*q & ADJACENCY) ) { // First adjacent
                    ADD_TO_EMPTYADJ(q);
                }
                score--; // New disk touches empty, which is bad
            }
        }
    } while (dir-- > 0);
}

```

```

    } else if (*q & enemy) {
        // Skip through line of enemy disks
        p = q + offset;
        while (*p & enemy)
            p += offset;

        if (*p & side) { // Flip 'em
            p = q;
            do {
                // Flip disk at p
                ++flips;
                if (IS_EDGE(*p))
                    score += kEdge;
                PUSH_SQ(p);
                score -= (COUNT_EMPTYIES(*p) << 1);
                // x2 Emptyies counted for us now
                *p ^= COLOR_BITS; // Flip
#ifdef USE_HASH
                gHashValue ^= *(p + gFlipHashOffset);
                // Update hash for flipped disk
#endif
                p += offset;
            } while (*p & enemy);
            score++;
            // Fills in area around disk at q which is now ours
        } else {
            score--;
            // Fills in area around disk at enemy disk at q
        }
        } else { // *q & side
            score++;
            // Fills in area around our disk at q
        }
        z = OPP_DIR_BIT(dir);
        MY_ASSERT((*q & z) == 0L);
        *q |= z;
    }
    } while (dir--);

    PUSH_SQ(to);
    REMOVE_FROM_EMPTYYADJ(to);
    *to |= side;

    PUSH(gCounts[WHITE_INDEX]);
    PUSH(gCounts[BLACK_INDEX]);
    gCounts[side >> COLOR_SHIFT] += flips + 1;
    gCounts[enemy >> COLOR_SHIFT] -= flips;

    if (IS_EDGE(*to))
        score += kEdge;

    return score;
}

UnmakeMove
void UnmakeMove()
{
    PSQUARE to, flipped, q;
    unsigned long z;
    long dir;

    // Restore disk counts
    gCounts[BLACK_INDEX] = POP;
    gCounts[WHITE_INDEX] = POP;

    // Replace to-disk
    to = (PSQUARE)POP;
    *to = POP;
    ADD_TO_EMPTYYADJ(to);

    // Undo disk changes
    while (POP) {
        flipped = (PSQUARE)TOP;

```

```

*flipped = POP;
}

// Update adjacency
dir = 7;
do {
q = to + gOffsets[dir];
z = OPP_DIR_BIT(dir);
*q &= ~z; // Remove adjacency (if not already removed from disk changes)
if ( IS_EMPTY(*q) && !(*q & ADJACENCY) ) { // First adjacent
REMOVE_FROM_EMPTYADJ(q);
}
} while (dir--);
}

```

Initialize

```
void Initialize(long boardSize, void *privStorage)
```

```

{
    unsigned long *p, *q;
    long i, index, numRealSquares;
    char *ptr;
    unsigned long z;
#ifdef USE_HASH
    unsigned long r1, r2;
#endif

    kX      = -2 - boardSize * 5;
    kC      = -2 - boardSize * 4;
    kEdge   = 3 + boardSize / 8;
    kCorner = 2 + boardSize * 13;

    gRealBoardSize = boardSize + 2;
    gNumOnSquares = boardSize * boardSize;
    numRealSquares = gRealBoardSize * gRealBoardSize;

    ptr = privStorage;

    gSquares = (unsigned long *)ptr;
    gOnBoardStart = gSquares + gRealBoardSize + 1;
    gOnBoardEnd   = gSquares + (gRealBoardSize * (gRealBoardSize - 1) -
2);
    ptr += numRealSquares * 4;
    // worst case 66*66*4 = 17,424 bytes (~17K)

#ifdef USE_HASH
    gWhiteHashOffset = numRealSquares;
    gBlackHashOffset = numRealSquares * 2;
    gFlipHashOffset  = numRealSquares * 3;
    ptr += (numRealSquares * 3) * 4;
    // worst case 66*66*3*4 = 52,272 bytes (~51K)

    gHashTable = (SHash *)ptr;
    ptr += kHashTableSize * sizeof(SHash);
    // 32768*16 = 524,288 bytes (512K)
#endif

    gEmptyAdj = (PSQUARE *)ptr;
    ptr += gNumOnSquares * 4;
    // worst case 64*64*4 = 16,384 bytes (16K)

    gChanges = (unsigned long *)ptr;
    ptr += 65536; // e.g. 64 moves (deep) * 256 longs/move * 4 bytes/long
    // 65,536 bytes (64K)

    gMobility = (long *)ptr;
    ptr += 1024; // e.g. 256 moves deep * 4 bytes/long
    // 1,024 bytes (1K)

    gCountZeros = (unsigned long *)ptr;
    ptr += 256 * 4;
    // 256*4 = 1,024 bytes (1K)

    gTree = (PSQUARE *)ptr;

```

```

// Gets what's left, almost 400K!

// ** Calculate directional offsets
gOffsets[DIR_S] = gRealBoardSize;
gOffsets[DIR_N] = - gRealBoardSize;
gOffsets[DIR_SE] = gRealBoardSize + 1;
gOffsets[DIR_NW] = - gRealBoardSize - 1;
gOffsets[DIR_NE] = - gRealBoardSize + 1;
gOffsets[DIR_SW] = gRealBoardSize - 1;

// ** Borders
// Upper/Lower
p = gSquares;
q = gSquares + (gRealBoardSize * (gRealBoardSize - 1));
i = gRealBoardSize;
do {
    *(p++) = BORDER_BIT;
    *(q++) = BORDER_BIT;
} while (--i);
// Sides
p = gSquares + gRealBoardSize;
i = gRealBoardSize - 2;
do {
    *p = BORDER_BIT;
    p += (gRealBoardSize - 1);
    *(p++) = BORDER_BIT;
} while (--i);

// ** Edges
// Upper/Lower
p = gOnBoardStart;
q = gOnBoardEnd;
i = boardSize;
do {
    *(p++) = NW_BORDER | N_BORDER | NE_BORDER;
    *(q--) = SW_BORDER | S_BORDER | SE_BORDER;
} while (--i);

// Sides
p = gOnBoardStart;
q = gOnBoardEnd;
i = boardSize;
do {
    *p |= NW_BORDER | W_BORDER | SW_BORDER;
    *q |= NE_BORDER | E_BORDER | SE_BORDER;
    p += gRealBoardSize;
    q -= gRealBoardSize;
} while (--i);

// ** Starting configuration
// Set up initial disks and adjacent empty squares
// "On your first move, you should initialize the board
// with white tiles at (row,col) = (boardSize/2-1,boardSize/2-1) and
// (boardSize/2,boardSize/2), and black tiles at (boardSize/2-1,boardSize/2)
// and (boardSize/2,boardSize/2-1)"
gCounts[WHITE_INDEX] = gCounts[BLACK_INDEX] = 2;

gSizeEmptyAdj = 12;
i = boardSize >> 1; // x2
index = XYZINDEX(i-1, i-1);

p = &gSquares[index];
*p = SE; gEmptyAdj[0] = p;
* (++p) = EMPTYADJ_BIT(1) | S | SE; gEmptyAdj[1] = p;
* (++p) = EMPTYADJ_BIT(2) | SW | S; gEmptyAdj[2] = p;
* (++p) = EMPTYADJ_BIT(3) | SW; gEmptyAdj[3] = p;

p += gRealBoardSize - 3;
*p = EMPTYADJ_BIT(4) | E | SE; gEmptyAdj[4] = p;
* (++p) = WHITE | S | SE | E;
* (++p) = BLACK | W | SW | S;
* (++p) = EMPTYADJ_BIT(5) | W | SW; gEmptyAdj[5] = p;

```

```

p += gRealBoardSize - 3;
*p = EMPTYADJ_BIT(6) | E | NE; gEmptyAdj[6] = p;
*(++p) = BLACK | N | NE | E;
*(++p) = WHITE | W | NW | N;
*(++p) = EMPTYADJ_BIT(7) | W | NW; gEmptyAdj[7] = p;

p += gRealBoardSize - 3;
*p = EMPTYADJ_BIT(8) | NE; gEmptyAdj[8] = p;
*(++p) = EMPTYADJ_BIT(9) | N | NE; gEmptyAdj[9] = p;
*(++p) = EMPTYADJ_BIT(10) | NW | N; gEmptyAdj[10] = p;
*(++p) = EMPTYADJ_BIT(11) | NW; gEmptyAdj[11] = p;

gNWCorner = gOnBoardStart;
gNWC1 = gNWCorner + 1; *gNWC1 |= BAD_BIT;
gNWC2 = gNWCorner + gRealBoardSize; *gNWC2 |= BAD_BIT;
gNWX = gNWC2 + 1; *gNWX |= BAD_BIT;

gNECorner = gNWCorner + boardSize - 1;
gNEC1 = gNECorner - 1; *gNEC1 |= BAD_BIT;
gNEC2 = gNECorner + gRealBoardSize; *gNEC2 |= BAD_BIT;
gNEX = gNEC2 - 1; *gNEX |= BAD_BIT;

gSWCorner = gOnBoardEnd - boardSize + 1;
gSWC1 = gSWCorner + 1; *gSWC1 |= BAD_BIT;
gSWC2 = gSWCorner - gRealBoardSize; *gSWC2 |= BAD_BIT;
gSWX = gSWC2 + 1; *gSWX |= BAD_BIT;

gSECorner = gOnBoardEnd;
gSEC1 = gSECorner - 1; *gSEC1 |= BAD_BIT;
gSEC2 = gSECorner - gRealBoardSize; *gSEC2 |= BAD_BIT;
gSEX = gSEC2 - 1; *gSEX |= BAD_BIT;

// Precalculate gCountZeros
// (Could have had the compiler fill these in, but I'm not
// THAT desperate for speed)
for (z=0; z<256; ++z) {
gCountZeros[z] =
8 - (z & 1) - ((z>>1) & 1) - ((z>>2) & 1) - ((z>>3) & 1) -
((z>>4) & 1) - ((z>>5) & 1) - ((z>>6) & 1) - ((z>>7) & 1);
}

#ifdef USE_HASH
gHashValue = 0xFFFFFFFF;

// Initialize gHashKeys
srand(0x1234); //srand(time(NULL));
for (i=0; i<numRealSquares; ++i) {
r1 = rand() + ((unsigned long)rand() << 16);
r2 = rand() + ((unsigned long)rand() << 16);
gSquares[gWhiteHashOffset + i] = r1;
gSquares[gBlackHashOffset + i] = r2;
gSquares[gFlipHashOffset + i] = r1 ^ r2;
}

// Clear gHashTable
{
SHash *pHashTable = gHashTable;

i = kHashTableSize - 1;
do {
pHashTable->HashValue = 0;
pHashTable->Depth = -100;
pHashTable->BestMove = NULL;
pHashTable->Type = INVALID;
pHashTable->Score = 0;
++pHashTable;
} while (i--);
}
#endif

gIncScore = 0;
}

```

SPREAD THE WORD:  [Slashdot](#)  [Digg](#)  [Del.icio.us](#)  [Reddit](#)  [Newsvine](#)

Generate a short URL for this page:

GET LINK

MacTech Magazine. www.mactech.com

Toll Free 877-MACTECH, Outside US/Canada: 805-494-9797

MacTech is a registered trademark of Xplain Corporation. Xplain, "The journal of Apple technology", Apple Expo, Explain It, MacDev, MacDev-1, THINK Reference, NetProfessional, Apple Expo, MacTech Central, MacTech Domains, MacNews, MacForge, and the MacTutorMan are trademarks or service marks of Xplain Corporation. Sprocket is a registered trademark of eSprocket Corporation. Other trademarks and copyrights appearing in this printing or software remain the property of their respective holders. Not responsible for typographical errors.

All contents are Copyright 1984-2011 by Xplain Corporation. All rights reserved. Theme designed by [lcreon](#).